

P1518R1

Stop overconstraining allocators in container deduction guides

Published Proposal, 2021-02-24

Authors:

[Arthur O'Dwyer](#)

[Mike Spertus](#)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

7

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1518-container-deduction-guides.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1518-container-deduction-guides.html

Abstract

Discussion of flatmap's deduction guides revealed that the deduction guides for sequence containers and container adaptors are needlessly overconstrained, making use cases such as pmr containers unnecessarily difficult. We fix this.

Table of Contents

- 1 Changelog
- 2 Stop overconstraining allocators that do not participate in deduction

- 2.1 The problem
- 2.2 The explanation
- 2.3 Proposed wording

3 Stop deducing from allocators that should not participate in deduction

- 3.1 The problem
- 3.2 The explanation
- 3.3 Proposed wording
- 3.4 Implementation note

References

Informative References

§ 1. Changelog

- R1: Incorporate LWG wording feedback. Mention [\[LWG3506\]](#) and [\[P1425R3\]](#).

§ 2. Stop overconstraining allocators that do not participate in deduction

§ 2.1. The problem

Consider this code:

```
std::pmr::monotonic_buffer_resource mr;
std::pmr::polymorphic_allocator<int> a = &mr;
std::pmr::vector<int> pv(a);

auto s1 = std::stack<int, std::pmr::vector<int>>(pv);
auto s2 = std::stack<int, std::pmr::vector<int>>(pv, a);
auto s3 = std::stack<int, std::pmr::vector<int>>(pv, &mr);

auto ds1 = std::stack(pv);
auto ds2 = std::stack(pv, a);
auto ds3 = std::stack(pv, &mr);
```

The initializers of `s1`, `s2`, and `s3` (which do not use CTAD) are all well-formed, as are the initializers of `ds1` and `ds2` (which do).

However, the natural and useful `ds3` is ill-formed, even though the `&mr` argument is irrelevant to determining the template arguments for `stack`! It seems clearly wrong that we give all the right information to unambiguously deduce the desired specialization of `stack` and then reject what would be a perfectly valid constructor invocation for that class. The allocator parameter's type does not *contribute* to class template argument deduction in these cases; it brings no new information, and therefore should be treated no differently than it is in the non-CTAD case. Indeed, we believe this is an oversight and should simply be fixed.

§ 2.2. The explanation

`stack` has one relevant deduction guide.

```
template<class Container, class Allocator>
stack(Container, Allocator) -> stack<typename Container::value_type, Co
```

This deduction guide satisfies the following constraints.

[container.adaptors.general] says:

A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true: ...

- It has an **Allocator** template parameter and a type that does not qualify as an allocator is deduced for that parameter.
- It has both Container and Allocator template parameters, and `uses_allocator_v<Container, Allocator>` is false.

To qualify as an allocator, a type `A` must have a nested typedef `A::value_type` ([container.requirements.general]/17). Since `std::pmr::monotonic_buffer_resource*` has no such nested typedef, `std::pmr::monotonic_buffer_resource*` doesn't qualify as an allocator. However, `uses_allocator_v<std::pmr::vector<int>, std::pmr::monotonic_buffer_resource*>` is `true`.

Because the type of `&mr` doesn't qualify as an allocator, the deduction guide drops out of overload resolution, failing deduction even though it has all the information needed to safely deduce the correct template arguments for `stack`.

§ 2.3. Proposed wording

To fix all three container adaptors, modify [\[container.adaptors.general\]](#) as follows:

A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true:

- It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
- It has a `Compare` template parameter and a type that qualifies as an allocator is deduced for that parameter.
- It has a `Container` template parameter and a type that qualifies as an allocator is deduced for that parameter.
- It has no `Container` template parameter, and it has an `Allocator` template parameter, and a type that does not qualify as an allocator is deduced for that parameter.
- It has both `Container` and `Allocator` template parameters, and `uses_allocator_v<Container, Allocator>` is false.

Note: The rationale is that if it does have a `Container` parameter, and the `Container` advertises that it can "use" the supplied allocator, we shouldn't interrogate the allocator parameter any more closely. Trust the `Container`'s specialization of `uses_allocator`.

Note: As of C++20, no container-adaptor deduction guide has an `Allocator` template parameter without also having a `Container` template parameter. But, as the resolution of [\[LWG3506\]](#) will add such deduction guides, LWG asked that we future-proof this wording.

Note: [\[P1425R3\]](#) also proposes new container-adaptor deduction guides which have an `Allocator` parameter without a `Container` parameter. (These guides were proposed in P1425R1, dropped in P1425R2, and restored in P1425R3.)

§ 3. Stop deducing from allocators that should not participate in deduction

§ 3.1. The problem

Consider the following code:

```
std::pmr::monotonic_buffer_resource mr;  
std::pmr::polymorphic_allocator<int> a = &mr;  
std::pmr::vector<int> pv(a);
```

```

auto v1 = std::vector<int, std::pmr::polymorphic_allocator<int>>(pv);
auto v2 = std::vector<int, std::pmr::polymorphic_allocator<int>>(pv, a);
auto v3 = std::vector<int, std::pmr::polymorphic_allocator<int>>(pv, &mr);

auto dv1 = std::vector(pv);
auto dv2 = std::vector(pv, a);
auto dv3 = std::vector(pv, &mr);

```

The initializers of `v1`, `v2`, and `v3` (which do not use CTAD) are all well-formed, as are the initializers of `dv1` and `dv2` (which do).

But the initializer of `dv3` is ill-formed!

Again, we know from the `pv` argument that the correct type for the vector is `decltype(pv)`, i.e. `std::pmr::vector<int>`. Therefore we know what is the possible range of types for the allocator parameter. The allocator parameter's type does not *contribute* to class template argument deduction in these cases; it brings no new information, and therefore should be treated no differently than it is in the non-CTAD case.

The problem we see with `vector` also occurs for all other sequence containers, associative containers, and unordered associative containers.

§ 3.2. The explanation

`vector` has only one deduction guide, and it's not relevant to what happens here. Here, we end up looking at the implicit guide generated from the constructor

```

template<class T, class Allocator>
class vector {
    vector(const vector<T, Allocator>&, const Allocator&);
};

```

From the first parameter, we deduce `T=int` and `Allocator=std::pmr::polymorphic_allocator<int>`. From the second parameter, we deduce `Allocator=std::monotonic_buffer_resource*`. We've deduced conflicting types for `Allocator`, so deduction fails.

In this case, the second argument unnecessarily participates in deduction, and again unexpectedly prevents natural and useful code from working as desired.

Observe that even in the absence of CTAD, the signature above permits construction only from the same specialization of `vector`, not from any arbitrary specialization of `vector`. For example:

```
std::vector<T, A1> v1;
std::vector<T, A2> v2(v1, A2()); // ill-formed
```

§ 3.3. Proposed wording

If we were introducing the containers from scratch today, the natural approach would be for the constructors to indicate which arguments should be considered deducible. LWG has indicated their satisfaction with this approach, so that's what we propose here.

Modify [\[deque.overview\]](#) as follows:

```
deque(deque&&);
deque(const deque&, const Allocator&);
deque(const deque&, const type_identity_t<Allocator>&);
deque(deque&&, const Allocator&);
deque(deque&&, const type_identity_t<Allocator>&);
deque(initializer_list<T>, const Allocator& = Allocator());
```

Modify [\[forwardlist.overview\]](#) as follows:

```
forward_list(forward_list&&);
forward_list(const forward_list&, const Allocator&);
forward_list(const forward_list&, const type_identity_t<Allocator>&);
forward_list(forward_list&&, const Allocator&);
forward_list(forward_list&&, const type_identity_t<Allocator>&);
forward_list(initializer_list<T>, const Allocator& = Allocator());
```

Modify [\[list.overview\]](#) as follows:

```
list(list&&);
list(const list&, const Allocator&);
list(const list&, const type_identity_t<Allocator>&);
list(list&&, const Allocator&);
list(list&&, const type_identity_t<Allocator>&);
list(initializer_list<T>, const Allocator& = Allocator());
```

Modify [\[vector.overview\]](#) as follows:

```
constexpr vector(vector&&) noexcept;  
constexpr vector(const vector&, const Allocator&);  
constexpr vector(const vector&, const type_identity_t<Allocator>&);  
constexpr vector(vector&&, const Allocator&);  
constexpr vector(vector&&, const type_identity_t<Allocator>&);  
constexpr vector(initializer_list<T>, const Allocator& = Allocator());
```

Modify [\[vector.bool\]](#) as follows:

```
constexpr vector(vector&& x);  
constexpr vector(const vector&, const Allocator&);  
constexpr vector(const vector&, const type_identity_t<Allocator>&);  
constexpr vector(vector&&, const Allocator&);  
constexpr vector(vector&&, const type_identity_t<Allocator>&);  
constexpr vector(initializer_list<bool>, const Allocator& = Allocator());
```

Modify [\[map.overview\]](#) as follows:

```
explicit map(const Allocator&);  
map(const map&, const Allocator&);  
map(const map&, const type_identity_t<Allocator>&);  
map(map&&, const Allocator&);  
map(map&&, const type_identity_t<Allocator>&);  
map(initializer_list<value_type>,  
    const Compare& = Compare,  
    const Allocator& = Allocator());
```

Modify [\[multimap.overview\]](#) as follows:

```
explicit multimap(const Allocator&);  
multimap(const multimap&, const Allocator&);  
multimap(const multimap&, const type_identity_t<Allocator>&);  
multimap(multimap&&, const Allocator&);  
multimap(multimap&&, const type_identity_t<Allocator>&);  
multimap(initializer_list<value_type>,  
    const Compare& = Compare,  
    const Allocator& = Allocator());
```

Modify [\[set.overview\]](#) as follows:

```

explicit set(const Allocator&);
set(const set&, const Allocator&);
set(const set&, const type_identity_t<Allocator>&);
set(set&&, const Allocator&);
set(set&&, const type_identity_t<Allocator>&);
set(initializer_list<value_type>, const Compare& = Compare(),
    const Allocator& = Allocator());

```

Modify [\[multiset.overview\]](#) as follows:

```

explicit multiset(const Allocator&);
multiset(const multiset&, const Allocator&);
multiset(const multiset&, const type_identity_t<Allocator>&);
multiset(multiset&&, const Allocator&);
multiset(multiset&&, const type_identity_t<Allocator>&);
multiset(initializer_list<value_type>, const Compare& = Compare(),
    const Allocator& = Allocator());

```

Modify [\[unord.map.overview\]](#) as follows:

```

explicit unordered_map(const Allocator&);
unordered_map(const unordered_map&, const Allocator&);
unordered_map(const unordered_map&, const type_identity_t<Allocator>&);
unordered_map(unordered_map&&, const Allocator&);
unordered_map(unordered_map&&, const type_identity_t<Allocator>&);
unordered_map(initializer_list<value_type> il,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());

```

Modify [\[unord.multimap.overview\]](#) as follows:


```

explicit unordered_multimap(const Allocator&);
unordered_multimap(const unordered_multimap&, const Allocator&);
unordered_multimap(const unordered_multimap&, const type_identity_t<Allocator>&);
unordered_multimap(unordered_multimap&&, const Allocator&);
unordered_multimap(unordered_multimap&&, const type_identity_t<Allocator>&);
unordered_multimap(initializer_list<value_type> il,
                    size_type n = see below,
                    const hasher& hf = hasher(),
                    const key_equal& eql = key_equal(),
                    const allocator_type& a = allocator_type());

```

Modify [\[unord.set.overview\]](#) as follows:

```

explicit unordered_set(const Allocator&);
unordered_set(const unordered_set&, const Allocator&);
unordered_set(const unordered_set&, const type_identity_t<Allocator>&);
unordered_set(unordered_set&&, const Allocator&);
unordered_set(unordered_set&&, const type_identity_t<Allocator>&);
unordered_set(initializer_list<value_type> il,
               size_type n = see below,
               const hasher& hf = hasher(),
               const key_equal& eql = key_equal(),
               const allocator_type& a = allocator_type());

```

Modify [\[unord.multiset.overview\]](#) as follows:

```

explicit unordered_multiset(const Allocator&);
unordered_multiset(const unordered_multiset&, const Allocator&);
unordered_multiset(const unordered_multiset&, const type_identity_t<Allocator>&);
unordered_multiset(unordered_multiset&&, const Allocator&);
unordered_multiset(unordered_multiset&&, const type_identity_t<Allocator>&);
unordered_multiset(initializer_list<value_type> il,
                    size_type n = see below,
                    const hasher& hf = hasher(),
                    const key_equal& eql = key_equal(),
                    const allocator_type& a = allocator_type());

```

§ 3.4. Implementation note

Some of the changes proposed in this paper have already been implemented "accidentally" by one or more library implementations, as shown in the following table and [this Godbolt](#):

<i>Construct</i>	<i>Well-formed</i>	<i>SFINAE-friendly ill-formed</i>	<i>Hard error</i>
<code>std::deque(pd, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::forward_list(pfl, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::list(pl, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::vector(pv, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::map(pm, &mr)</code>	MSVC, libc++	libstdc++	
<code>std::multimap(pmm, &mr)</code>	MSVC, libc++	libstdc++	
<code>std::multiset(pms, &mr)</code>	MSVC, libc++	libstdc++	
<code>std::set(ps, &mr)</code>	MSVC, libc++	libstdc++	
<code>std::unordered_map(pum, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::unordered_multimap(pumm, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::unordered_multiset(pums, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::unordered_set(pus, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::priority_queue(ppq, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::queue(pq, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::stack(ps, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::priority_queue(comp, pv, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::queue(pd, &mr)</code>		MSVC, libstdc++, libc++	

`std::stack(pv, &mr)`

MSVC, libstdc++,
libc++

§ References

§ Informative References

[LWG3506]

Tim Song. [LWG 3506: Missing allocator-extended constructors for priority_queue](https://cplusplus.github.io/LWG/issue3506). November 2020. URL: <https://cplusplus.github.io/LWG/issue3506>

[P1425R3]

Corentin Jabot. [P1425R3: Iterator-pair constructors for stack and queue](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1425r3.pdf). February 2021. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1425r3.pdf>